

Section 2. Technical project details and outcomes

I. Outline

Financial data is often limited in quantity which makes it difficult for neural networks to improve their forecasts. Synthetic data helps in capturing the potential patterns and trends apart from increasing the sample points in the dataset. This is very useful for time series analysis as the patterns and trends may not be evident or obvious at first glance. Here, we aim to create a wholesome dataset by using different augmentation methods which can provide a more accurate representation of the underlying trends and patterns in the original dataset. We propose a solution which entails the use of a simple trading strategy in conjugation with different augmentation methods which can improve our portfolio returns in the long-term. These augmentation methods can help create a dataset which is more diverse, comprehensive and reflective of the real-world market. Our research can help to reduce the risk of financial decision making while also improving the returns from the financial forecasting models.

Given below is a brief overview of our project. Our main focus in the project is the data augmentation part. We improve the time series forecast model with augmentation, aiming to improve financial performance significantly when combined with a specific trading strategy. In the end, we can utilize the best performing techniques to create a portfolio.

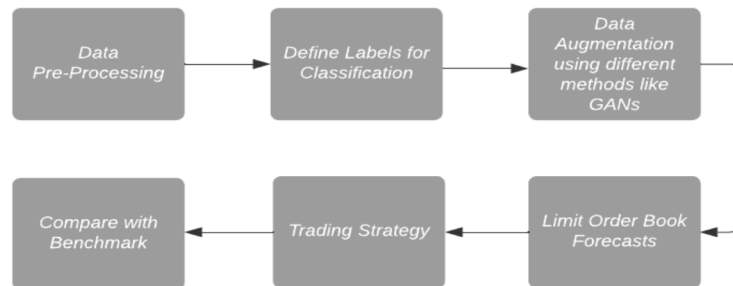


Figure 1: Proposed milestones in the project

II. Related work

We are building on the works of data pre-processing techniques as described by Krauss et al. [11] and Elizabeth et al. [12]. For the augmentation and the generative models, we are using some of the techniques mentioned in section IV-D. Apart from that we can also utilize and build on the work of Koshiyama et al(2019)[13] whose GAN-based approximation of a conditional model and definition of a selection score for simulator and parameter candidates can give interesting results. Pardo (2019)[14] shows synthetic pathways with volatility clusters and

related statistics using Wasserstein and relativistic GANs. Additionally, they show how the usage of synthetic pathways can improve a trading strategy's accuracy (Pardo and Lopez, 2019[15]) which can be further used for improving accuracy in our case. By utilizing adversarial training and discrete local volatilities, Wiese et al. (2019a)[16] demonstrated how recurrent architectures can be used to create stock option markets that adhere to static arbitrage requirements. For the trading strategy, we plan to utilize a simple rule based strategy as followed by Elizabeth et al. [12]. Advanced work can also include a momentum based trading strategy as discussed in Daniel et al [17].

III. Methodology

The methodology used in this research consists of several steps. Firstly, we are collecting the historical stock price data and preprocessing it to make it usable. Then, we perform feature analysis on the collected data to add a couple of other features. Post this step, we use various augmentation methods to generate synthetic data. The augmented data is further split into training and validation sets. Then we use the former part to train out machine learning models like neural network models (LSTM). The trained model is then evaluated on the basis of augmented validation dataset and test sets. We use performance metrics like accuracy, precision, recall, and F1 score to check the performance of our models. Post that we use a simple trading strategy to calculate returns. The performance of these returns are measured by a couple of financial performance indicators which we discuss later. These parameters are then compared to determine the effectiveness of each data augmentation method in improving stock forecasting accuracy. Finally, a conclusion is drawn based on the analysis of the results and recommendations are provided for the most effective data augmentation methods for stock forecasting. More details for each of the step is given below:

A. Data collection:

We are implementing our research in the stock market domain. We are utilizing the constituent stocks of the S&P 500 index. Later, we plan to implement the selected strategy to other stock exchanges as well and check the performance of the trading strategy there. We used Yahoo Finance API to download stock data. We can find data about the constituents of S&P 500 from some open source project links. [9] [10]

B. Pre-processing

We are following the data pre-processing methods from Krauss et al. [18]. There are following main parts involved in the complete steps:

1. Study period division: We are first dividing the data into small buckets or study periods of 1000 days and applying different data augmentation methods to each bucket. Each study period overlaps the other by 750 days. By dividing the data into small buckets and applying different data augmentation methods to each

bucket, we are able to mitigate the effects of data sparsity and generate more robust training data.

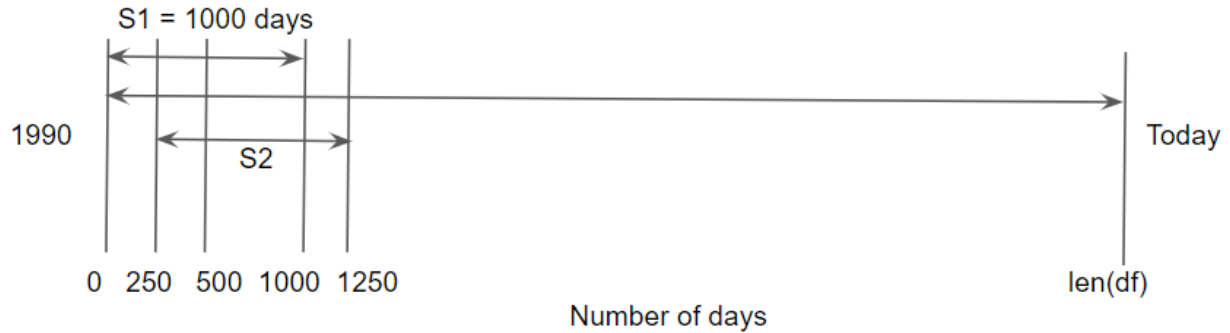


Figure 2: Breaking down entire dataframe into small study periods to remove data sparsity problems

2. Studying selected stocks: After downloading the returns for all the stocks, the first step is to select only the companies that existed at the beginning of the testing period.

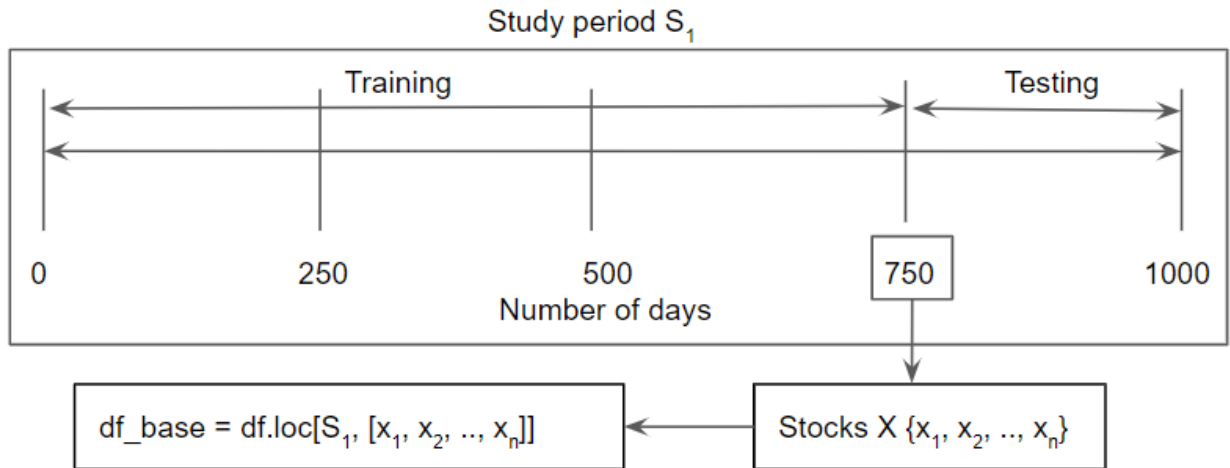


Figure 3: For each study period S_k , we only use stocks that are present in the testing period.

3. Preparing independent variable (df_X): We are preparing our input independent variable dataframe by implementing a Z-Score Standardization for each stock s which gives us R_t^s the return of stock s at time t . So, for each study period we take returns from all the stocks in the training period, and calculate their mean & standard deviation. This process normalizes the data in the dataframe so that it is scaled with a mean of 0 and a standard deviation of 1. We perform this process

because the stock price data usually has large variations over time and also, different stocks can have different levels of volatilities. The process of normalization helps us to standardize the returns data so that we can compare different stocks over different time periods. We center the data around 0 by subtracting the mean from each data point to remove any inherent bias in the data. Then, for scaling we divide the data by the standard deviation to have a standard deviation of 1. This method is useful because it allows us to compare the volatility of different stocks and time periods on a relative basis.

```

μ= df_base[:750].mean()
σ= df_base[:750].std()
df_X = (df_base - μ)/σ

```

Figure 4: Preparing independent variable df by normalizing base_df for each study period, S_k

4. Cleaning (df_base) for dependent variable (df_Y_cleaned): We prepare our input dependent variable dataframe by first cleaning for NaNs/blanks. For each study period, we take the input data frame df_base and create a copy of it called df_Y_cleaned. Then we iterate over all the stocks and over the rows from the 240th row until the end of the dataframe (1000th day). For each row, we check whether the previous 240 rows (i.e. the previous 240 days) for that stock contain any missing values (NaN) in the original dataframe, df_base. If there are missing values, it means that there isn't enough history to calculate the returns, so the function replaces the return value for that row and column with NaN in the df_Y_cleaned.

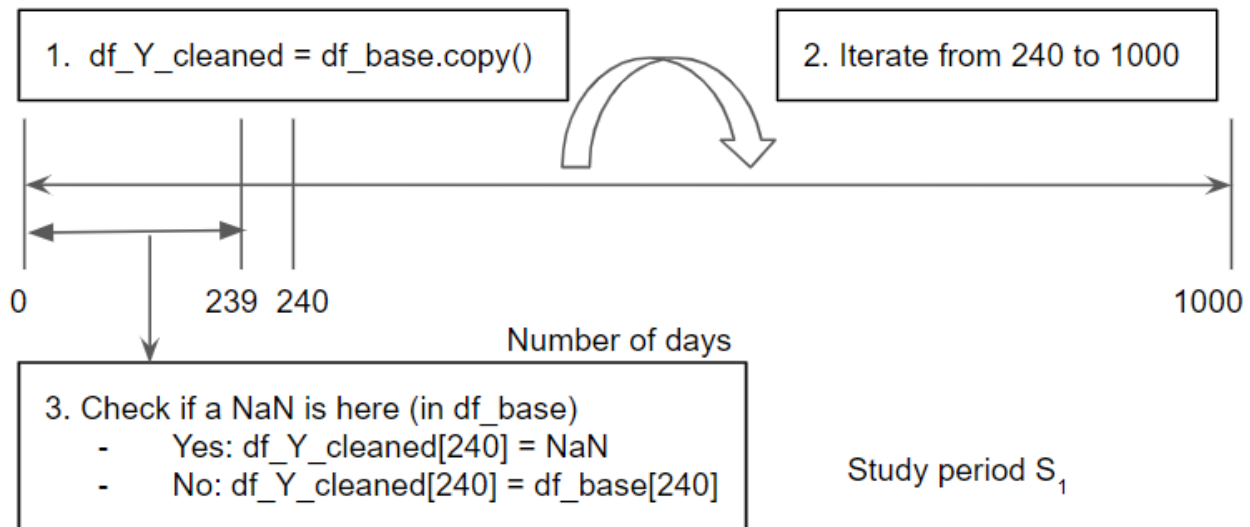


Figure 5: Cleaning dataframe of NaNs; must have 240 non-null prior values.

5. Labeling for dependent variable (df_Y): We are presently using Elizabeth et al.[12]'s method which defines a target variable Y_{t+1}^s which is equal to 1 if the returns are above the daily median and 0 if returns are below the daily median.

Algorithm 1 Labeling

```

if  $Y_s^{t+1} > Y_{median}^{t+1}$  then
     $Label = 1$ 
else
     $Label = 0$ 
end if

```

The target variable is Y_s^{t+1} and the daily median is Y_{median}^{t+1} .

Figure 6: Labeling df_Y using median returns for each day.

6. Slicing dataframe for testing and training: In the next step we slice the df_X and df_Y dataframe for testing and training. For each label in df_Y, we are taking prior 240 days historical values from df_X. For the training set, we are iterating from day 241 to day 750; while for testing, we are iterating from day 750 to day 1000 with a lookback of 240 days as earlier stated.

C. Augmentation

For data augmentation methods, we are implementing some of the different techniques which have been discussed in previous research works. Some of the augmentation techniques that we are implementing are listed below:

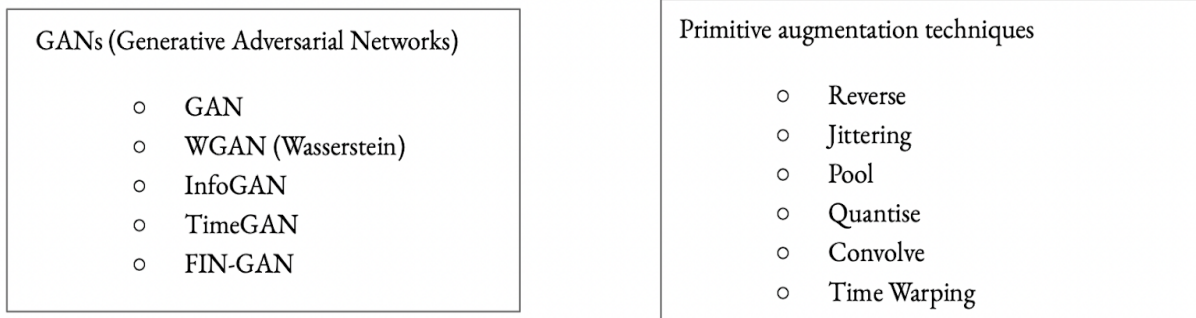


Figure 7: Generative modeling techniques that we plan to use

We are splitting the training data into training and validation sets. Data augmentation is then further carried out in the shuffled sample by augmenting each training set with 1X

the original size. In this part, we have completed the primitive techniques as well as a simple-GAN. We have also done more advanced methods like TimeGAN and InfoGAN, that we will present.

1. InfoGAN

In traditional GANs, the generator uses the method of mapping random noise to realistic-looking outputs to generate samples, but it cannot ensure that the latent variables correspond to any specific semantic features of the input and it is difficult to modify existing samples in a meaningful way later. The infoGAN adds an additional input on the basis of traditional GAN, to eliminate the effect of changes to one latent variable on multiple aspects of the generated output.

The key idea of infoGAN is maximizing the mutual information between the latent code and generated examples. Therefore, we chose infoGAN to deal with the original training datasets.

-The overall objective function: $\min_G \max_D V(D, G) + \lambda I(c; G(z, c))$

-The estimating formula of mutual information:

$$I(c; G(z, c)) = E[\log(q(c|G(z, c))) - \log(p(c))]$$

With the infoGAN method, we generated synthetic stock market data and successfully augmented existing datasets for training other machine learning models (LSTM and XGBoost). Our results indicate a notable increase in the performance of predictive models, with improvements of 14.6% and 21% respectively compared to the benchmark without data augmentation, which can be applied to stock price forecasting and portfolio optimization.

2. TimeGAN

The traditional generative adversarial networks don't capture the temporal dynamics of the time-series data. The autoregressive models can factor the distribution of sequences but are fundamentally deterministic in the context of forecasting. Yoon et al. [19] propose a TimeGAN framework that combines the unsupervised GAN framework with the control afforded by supervised training in an autoregressive model, which achieves consistent better results than the benchmark. Therefore, we further use the TimeGAN to capture the temporal dynamics of the training datasets.

Autoregressive decomposition: $p(\mathbf{S}, \mathbf{X}_{1:T}) = p(\mathbf{S}) \prod_t p(\mathbf{X}_t | \mathbf{S}, \mathbf{X}_{1:t-1})$

Our model has the following two objectives:

- GAN objective: $\min_{\hat{p}} D\left(p(\mathbf{S}, \mathbf{X}_{1:T}) \parallel \hat{p}(\mathbf{S}, \mathbf{X}_{1:T})\right)$

- Maximum-likelihood objective: $\min_{\hat{p}} D\left(p(\mathbf{X}_t | \mathbf{S}, \mathbf{X}_{1:t-1}) \parallel \hat{p}(\mathbf{X}_t | \mathbf{S}, \mathbf{X}_{1:t-1})\right)$

D. Training Base Model

The original data is then concatenated with the augmented dataset and fed into a machine learning model. We are presently using a deep learning model called LSTM and get an accuracy of 51.38%. LSTM (Long Short-Term Memory) neural network is a type of recurrent neural network (RNN) architecture that is designed to handle the vanishing gradient problem in traditional RNNs. We plan to tune the parameters of the present model to get even more effective results. We also use a machine learning algorithm called XGBoost and get an accuracy of 49.59%, which is not very ideal. Next, We plan to try out more different algorithms like decision tree, random forests to improve our results.

IV. Evaluation

The evaluation of the data augmentation methods can be further performed by looking at long term returns. We evaluate different methods by using a simple trading strategy. More details for each step are given below:

A. Benchmark and Baseline

We build our portfolio by taking top n-stocks in the S&P 500 index where we can tweak the number of stocks (n) to 5,10, 50,100,150, 200,etc. Then, we make a benchmark of all stocks in the market. We keep these stocks by equal weights. Furthermore, we build all the portfolios inclusive of the transaction costs. A baseline index is also created which measures the returns if no data augmentation techniques were used for the forecasts.

B. Trading Strategy

In order to evaluate the different data augmentation strategies in asset allocation, we build on the works of Elizabeth et al [12] which followed a simple rule based trading strategy. The strategy ranks stocks in both classes daily by their predicted probability of belonging to that class, and then recommends taking the top 10 and bottom 10 stocks and building a long-short portfolio by equally weighting the stocks. Portfolios are further analyzed after transaction costs of 5 bps per trade. We can also explore some other strategies like a momentum based one as explained by Daniel et al[17].

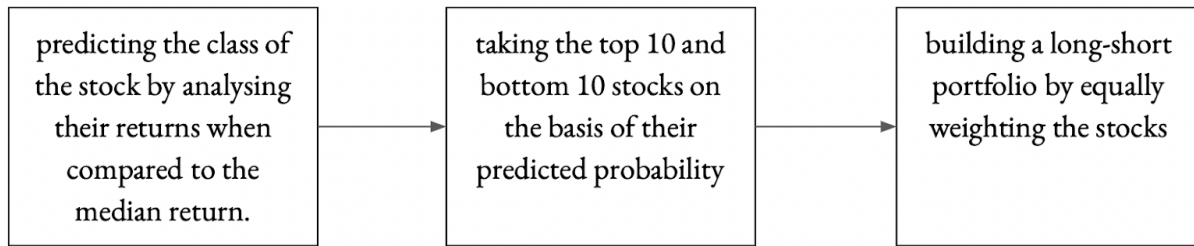


Figure 3: Proposed trading strategy by Elizabeth et al

C. Metrics of success

We are using a few financial metrics to evaluate and measure the performance of our trading strategy. We are listing some of the metrics that we are using here. There are more metrics that we have measured as discussed later in the results section of the report.

1. **Return:** We express returns as a percentage which shows the total gains (or losses) by using the strategy in the given time period. We can determine the efficacy of our strategies by comparing the portfolio performance with the benchmark returns.
2. **Sharpe Ratio:** This metric measures the portfolio's return while taking into account both returns and the risk of the portfolio. A higher Sharpe ratio means the portfolio has done decently well given the risk it carries. We can compare the Sharpe ratio of the portfolio to a benchmark to determine the risk-adjusted return of the portfolio.
3. **Tracking Error:** This metric provides us the volatility of portfolio's excess returns when compared to a benchmark. It is calculated by using standard deviation of portfolio's excess return over a specified time period.
4. **Information Ratio:** This metric shows the portfolio's risk-adjusted return in relation to a benchmark index. It is calculated by dividing the portfolio's excess return by the portfolio's tracking error. A high IR shows an outperforming portfolio.

V. Results

We plot the results of our research here in this section. First let's explore the cumulative excess returns for various augmentation methods when compared to market benchmark (Figure 8).

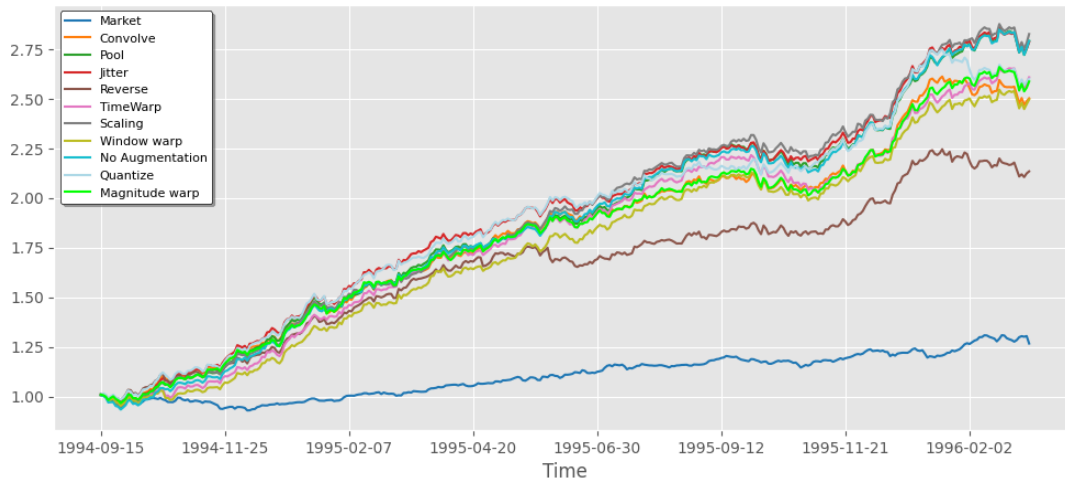


Figure 8: Cumulative excess returns for a specific study period.

Now, let's try and plot a heatmap for various metrics that we are measuring for different augmentation methods (Figure 9).

	Calmar Ratio	Information Ratio	Tracking Error	Max Drawdown	Sortino Ratio	Sharpe Ratio	Annualized Volatility	Annualized Return
Market	4.182383			-0.040398	0.158635	1.752869	0.089209	0.168959
Convolve	21.7994	0.196159	0.009417	-0.03948	0.500339	4.489784	0.138651	0.860632
Pool	25.756516	0.22609	0.009479	-0.038988	0.578755	4.949001	0.140864	1.004201
Jitter	31.250157	0.224414	0.009532	-0.032071	0.564084	4.895275	0.142207	1.00221
Reverse	15.587502	0.15014	0.009511	-0.043271	0.374377	3.657935	0.14126	0.674479
TimeWarp	15.753663	0.207391	0.009466	-0.058119	0.503518	4.575414	0.142443	0.915587
Scaling	30.114477	0.233843	0.009332	-0.034003	0.622023	5.171851	0.136702	1.023982
Window warp	18.838674	0.199814	0.009258	-0.045748	0.495038	4.512718	0.13809	0.861827
No Augmentation	25.040398	0.229477	0.009371	-0.040254	0.60018	5.001841	0.139754	1.007971
Quantize	18.911849	0.206663	0.009418	-0.04799	0.521922	4.62341	0.140054	0.907577
Magnitude warp	21.23648	0.209283	0.009263	-0.042562	0.519089	4.648257	0.138885	0.903863

Figure 9: Cumulative performance heatmap for various data augmentation methods.